

HDI INSIGHT – HUMAN DEVELOPMENT INDEX PREDICTOR

Machine Learning-Powered HDI Prediction & Analysis Platform

Project Description:

In today's data-driven world, understanding global human development is crucial for policymakers, researchers, and organizations. Traditional methods of analyzing the Human Development Index (HDI) often involve manual calculations and static reports, which lack interactivity and predictive capabilities.

With the advancement of machine learning and web technologies, there is a strong need for a centralized digital platform that simplifies and enhances HDI analysis. HDI Insight – Human Development Index Predictor is developed to address these challenges by providing a secure, scalable, and user-friendly web-based solution for predicting HDI scores and classifying countries into development tiers.

The primary objective of the HDI Insight system is to bridge the gap between complex data science and user accessibility by offering a digital ecosystem where HDI analysis can take place efficiently and interactively. The platform enables users to input key socio-economic indicators, including life expectancy, mean years of schooling, expected years of schooling, and GNI per capita. On the other hand, the system processes these inputs using a trained machine learning model to output an accurate HDI score and categorize the country into Very High, High, Medium, or Low development tiers.

Unlike traditional analysis tools that provide limited features, HDI Insight offers:

- Secure user authentication and role-based access
- Real-time machine learning predictions
- Structured database-driven prediction history
- AI chatbot (Gemini) for data insights
- Stripe integration for credit payments
- Premium blog access for advanced insights
- Modular and scalable microservices architecture

The system follows a modern microservices development approach using React (Vite) for the frontend interface, Node.js and Express for the backend API handling, FastAPI for the machine learning service, and MongoDB Atlas for database management. This architecture ensures separation of concerns, improved performance, and easier maintainability.

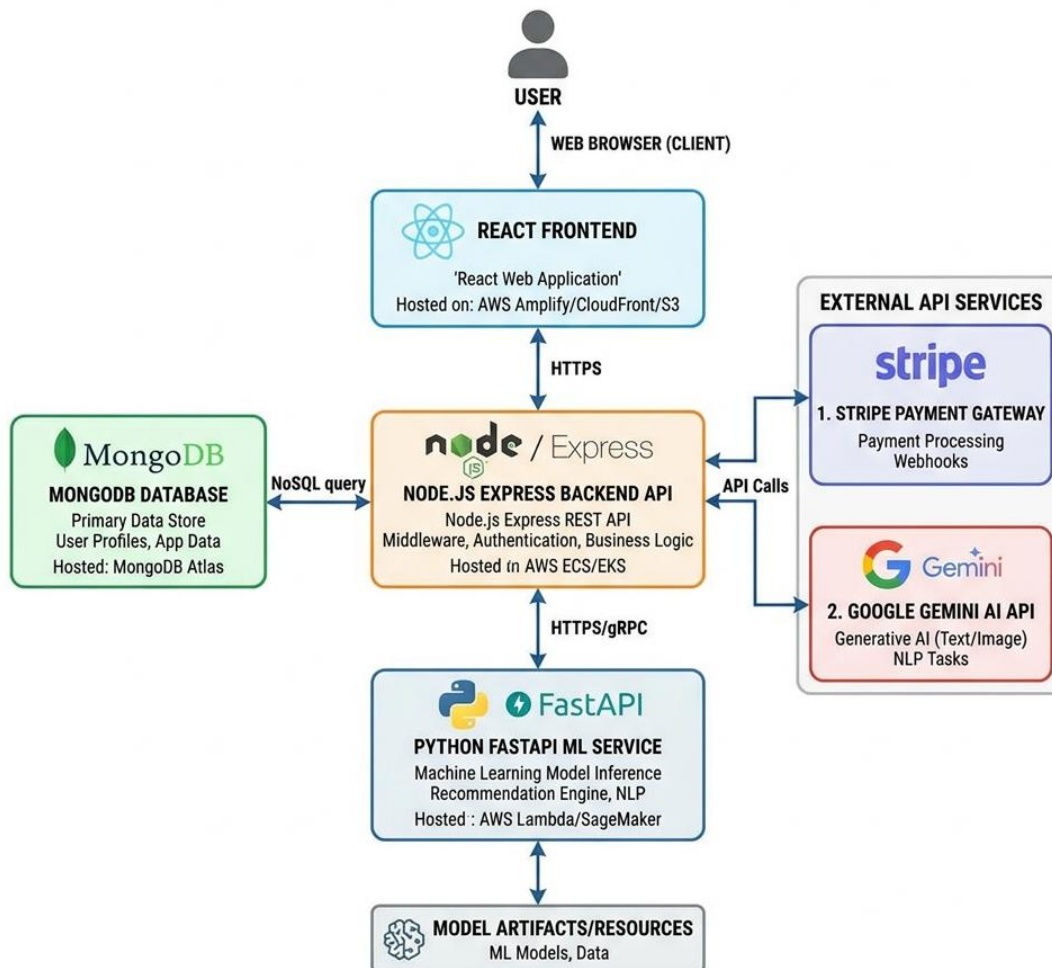
One of the key strengths of the HDI Insight system is its robust user management and payment integration. The application supports primary user interactions:

1. Users – Can predict HDI scores, view their prediction history, interact with the AI chatbot, and purchase credits.
2. Premium Users – Gain access to exclusive premium blog content and advanced features.

By implementing secure authentication using JSON Web Tokens (JWT) and password encryption, the system ensures user data confidentiality and protection against unauthorized access. The system is also designed with scalability in mind. As the number of users and prediction requests increases, the microservices architecture and MongoDB Atlas database organization allow smooth handling of larger datasets without performance degradation. Furthermore, the Stripe payment integration enhances the platform by allowing users to seamlessly purchase prediction credits, complete with payment success and cancellation workflows.

Technical Architecture:

MODERN WEB APPLICATION ARCHITECTURE DIAGRAM



Pre requisites:

To successfully develop, test, and deploy the HDI Insight system, certain software tools, programming knowledge, and packages are required. These prerequisites ensure smooth development, proper backend–frontend integration, database connectivity, secure authentication, and efficient machine learning implementation.

Software Required

- **Node.js**

Node.js is a JavaScript runtime environment used to build the Express backend server for handling API requests, authentication, and database communication.

- **Python**

Python is used for the FastAPI ML service and running scikit-learn machine learning models.

- **MongoDB Atlas**

MongoDB is a NoSQL database used to store application data such as users and prediction history.

- **Visual Studio Code (VS Code)**

The primary code editor used for writing frontend and backend code.

- **Postman**

Used for testing backend APIs.

- **Git**

Used for version control.

- **Web Browser (Chrome Recommended)**

Used for frontend testing and debugging.

Backend Dependencies (Node.js & FastAPI)

- **express**

Lightweight web framework for Node.js.

- **mongoose**

ODM library for MongoDB.

- **jsonwebtoken (JWT)**

Used for secure authentication.

- **stripe**

Used for payment processing integration.

- **fastapi**

Python framework for building the ML API.

- **scikit-learn, pandas, numpy**

Python libraries for machine learning data processing and prediction.

- **joblib**

Used for loading the trained ML model.

- **google-generativeai**

Gemini API integration for the AI chatbot.

- **cors**

Enables Cross-Origin Resource Sharing.

- **dotenv**

Used to manage environment variables.

Frontend Dependencies

- **react**

Core library for building user interfaces.

- **react-router-dom**

Used for navigation and routing.

- **axios**

Used for making HTTP requests to backend APIs.

- **tailwindcss / CSS**

Used for styling and responsive design.

- **lucide-react / react-icons**

Provides scalable icons.

- **react-toastify**

Displays notifications and alerts.

- **@stripe/stripe-js**

Frontend Stripe integration for secure checkouts.

Prior Knowledge:

You must have prior knowledge of the following topics to successfully complete the HDI Insight project.

- Full Stack Development Concepts (React Basics, Components, React Hooks, React Router, Axios)
- Backend Development (Node.js, Express, FastAPI, REST API Concepts, Middleware, Microservices)

- Machine Learning Concepts (scikit-learn, Model Training, Predictions)
- Database Concepts (MongoDB, Mongoose, CRUD Operations, Schema Design)
- Authentication & Security (JWT, Password Hashing, Stripe Payment Flow)
- Additional Concepts (HTTP Methods, JSON, Generative AI APIs, Git)

Project Objectives:

By the end of this project, you will:

- Understand the fundamentals of microservices web application development.
- Gain practical knowledge of frontend development using React.js (Vite).
- Learn how to design and implement RESTful APIs using Node.js, Express, and FastAPI.
- Develop the ability to train and deploy machine learning models using scikit-learn and joblib.
- Understand how to integrate third-party APIs like Stripe for payments and Google Gemini for AI chat.
- Learn how to build secure authentication using JWT and password hashing.
- Implement CRUD operations for prediction history and user management.
- Develop dynamic machine learning prediction workflows.
- Understand the integration between multiple backend services and a frontend interface using Axios.
- Gain experience in modular project structuring using microservices architecture.
- Learn how to deploy a multi-service application on Render.

Project Flow:

- The user interacts with the React-based User Interface (UI).
- The frontend collects user input such as registration details, login credentials, HDI indicators, or payment requests.
- The frontend sends HTTP requests to the Express backend APIs using Axios.
- For predictions, the Express backend communicates with the FastAPI ML service.
- The FastAPI service loads the scikit-learn model, processes the input, and returns the predicted HDI score and category.
- Authentication is verified using JWT tokens.
- The backend communicates with MongoDB Atlas database using Mongoose to save prediction history and user credits.
- External API calls are made to Stripe (for payments) and Gemini (for AI chatbot).
- The backend sends a JSON response back to the frontend.
- The frontend dynamically updates the UI and displays the result to the user.

To accomplish this, the following activities are performed:

- Requirement Analysis (Features, ML parameters)

- Backend Development (Express API, FastAPI ML API, MongoDB setup, JWT, Stripe, Gemini)
- Frontend Development (React app, UI design, Stripe checkout, Axios integration)
- Machine Learning (Data preprocessing, Model training, Exporting via joblib)
- Testing & Debugging (Postman, UI validation, Integration testing)
- Deployment (Render deployment, Environment variables setup)

Project Structure:

Create the Project folder which contains files as shown below:

HDI-INSIGHT-MAIN

```

|
|  └─ client
|    |  └─ node_modules
|    |  └─ public
|    |  └─ src
|    |    |  └─ assets
|    |    |  └─ components
|    |    |  └─ pages
|    |    |  └─ App.css
|    |    |  └─ App.jsx
|    |    |  └─ index.css
|    |    |  └─ main.jsx
|    |  └─ .env
|    |  └─ package.json
|    |  └─ vite.config.js
|    |
|    └─ server
|      |  └─ controllers
|      |  └─ database
|      |  └─ middlewares
|      |  └─ models
|      |  └─ routes
|      |  └─ utils
|      |  └─ .env
|      |  └─ index.js
|      |  └─ package.json
|      |
|      └─ ml-service
|        |  └─ model

```

```
| | └─ hdi_model.pkl
| └─ main.py
| └─ requirements.txt
```

Backend Folder:

The backend consists of two services: the Node.js Express server and the Python FastAPI service.

Node.js Express Server:

- **database folder**

Contains MongoDB database connection configuration files.

- **controllers folder**

Contains business logic files (auth, predictions routing, payments, chat).

- **middlewares folder**

Contains authentication and authorization middleware such as JWT verification.

- **routes folder**

Defines REST API endpoints (/api/auth, /api/predict, /api/payment, /api/chat).

- **models folder**

Contains Mongoose models (User, PredictionHistory).

- **utils folder**

Contains helper utilities.

- **.env file**

Stores environment variables (MongoDB URI, JWT Secret, Stripe Keys, Gemini API Key).

- **index.js**

Main entry file connecting to the database and starting the Express server.

Python FastAPI Service:

- **model folder**

Contains the trained hdi_model.pkl generated by joblib.

- **main.py**

Entry point for the FastAPI server exposing the /predict endpoint.

- **requirements.txt**

Python dependencies for the ML service.

Frontend Folder:

The frontend folder (client) contains React application files.

- **public folder**

Contains static files and index.html template.

- **src folder**

Contains React components, features, and pages (Login, Signup, Predict, History, Chat, Premium Blog).

- **App.jsx**

Main React application component.

- **main.jsx**

Entry point of the React application.

- **package.json**

Contains frontend dependencies and scripts.

- **vite.config.js**

Configuration file for Vite build tool.

Milestone 1: Requirement Analysis

Software development begins with proper requirement analysis. In this phase, the system requirements of HDI Insight are identified, analyzed, and documented clearly.

Activity 1: Identifying System Requirements

- User Registration and Login
- Machine Learning HDI Prediction
- AI Chatbot Integration (Gemini)
- Payment Gateway Integration (Stripe)
- Prediction History Tracking
- Premium Blog Access

Activity 2: Functional Requirements

- Users must be able to register and log in securely.
- Users must be able to input 4 indicators and receive an HDI prediction.
- Users must be able to view their past predictions.
- Users must be able to purchase credits via Stripe.
- Users must be able to interact with an AI chatbot for insights.
- Premium users must have access to exclusive blog content.

Activity 3: Non-Functional Requirements

- System must be secure and scalable.
- Response time for ML predictions must be fast.
- UI must be user-friendly and responsive.
- System must handle multiple microservices communication efficiently.
- Data must be protected using encryption and secure authentication mechanisms.

Milestone 2: Backend & ML Development

After requirement analysis, backend and ML development is initiated.

Activity 1: Project Setup

- Install Node.js & Python
- Initialize npm and pip environments
- Install required dependencies (Express, FastAPI, scikit-learn, etc.)
- Configure environment variables

Activity 2: Database Connection

- Setup MongoDB Atlas
- Configure Mongoose
- Connect database to Express server

Activity 3: ML Model Training & API

- Train HDI prediction model using scikit-learn on UNDP dataset
- Export model using joblib
- Create FastAPI endpoint to load model and serve predictions

Activity 4: Creating Schemas

- User Schema (Credits, Premium status)
- Prediction History Schema

Activity 5: Implementing Authentication & APIs

- Hash passwords using bcrypt
- Generate JWT tokens
- Build REST APIs for auth, payments (Stripe checkout), and chat (Gemini)
- Implement middleware to protect routes

Milestone 3: Frontend Development

After backend APIs are ready, frontend development begins.

Activity 1: Create React Application

- Setup React project using Vite
- Install required dependencies
- Configure routing using React Router

Activity 2: Design User Interface

- Home Page / Prediction Interface
- Login / Signup Page
- Prediction History Dashboard
- AI Chatbot Interface
- Stripe Payment Success / Cancel Pages
- Premium Blog Page

Activity 3: API Integration

- Use Axios to connect frontend to Express backend
- Send authentication tokens in headers
- Display dynamic prediction results and chat responses

Milestone 4: Testing & Debugging

Testing ensures the system works correctly.

Activity 1: Backend Testing

- Test Node.js and FastAPI endpoints using Postman
- Verify Stripe webhooks and Gemini API limits
- Validate error handling

Activity 2: Frontend Testing

- Test form validation (HDI indicators)
- Verify UI responsiveness
- Test protected routes and Stripe redirects

Activity 3: Integration Testing

- Ensure seamless communication between React -> Express -> FastAPI
- Confirm database operations and credit deductions

Milestone 5: Deployment & Finalization

Activity 1: Code Optimization

- Optimize ML model loading
- Improve API performance

- Remove unnecessary code

Activity 2: Deployment Preparation

- Deploy Express backend, FastAPI service, and React frontend on Render (<https://hdi-insight.onrender.com>)
- Configure environment variables on Render
- Test final deployed application

Activity 3: Documentation

- Prepare project report
- Add architecture diagrams and screenshots

Application Screens & Output Explanation

1. User Registration Page (Sign Up Interface)

[Note: Insert screenshot of HDI Insight Sign Up page here]

This page allows new users to create an account by entering Full Name, Email Address, and Password. Securely hashes password and creates MongoDB record.

2. Home Page – HDI Prediction Interface

[Note: Insert screenshot of HDI Insight Prediction page here]

Users enter the 4 key indicators: Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita. The Express backend routes this to the FastAPI service, which returns the HDI score and classification (e.g., Very High). Credits are deducted upon successful prediction.

3. Prediction History Dashboard

[Note: Insert screenshot of HDI Insight History page here]

Displays a tabular record of the user's past predictions retrieved from MongoDB.

4. AI Chatbot Interface

[Note: Insert screenshot of HDI Insight Gemini Chatbot page here]

An interactive chat interface where users can ask questions about human development, powered by the Google Gemini API.

5. Stripe Payment Interface & Premium Blog

[Note: Insert screenshot of Stripe Checkout and Premium Blog pages here]

Users can securely purchase credits via Stripe. Upon success, they are redirected to a Success page, updating their credit balance and unlocking Premium Blog content.

UI Design & User Experience

The interface follows a modern and minimal design:

- Clean navigation bar
- Responsive layouts for forms and chat
- Smooth Stripe checkout flow
- Dark mode feature

Ensuring ease of use and fast loading performance.

System Output Summary

The HDI Insight system successfully demonstrates:

- Secure user authentication
- Real-time machine learning predictions via FastAPI
- Microservices architecture integration
- Stripe payment gateway functionality
- Generative AI chatbot integration
- Database-driven prediction history